

CX 4220/CSE 6220 High Performance Computing

PA 2 Report

Submitted March 7, 2025

Group Member: Xuzheng Tian & Honglin Liu

1. AllReduce algorithm:

To achieve hypercubic communication, we create a for loop of variable d :

- (a) Apply bitwise left shift operator $1 \ll d$ to represent the gaps between communicate processors. For loop breaks when the gap is larger than the total processors.
- (b) Inside the loop, obtain the partner processor for the current rank, using XOR operator $partner = rank \text{ xor } (1 \ll d)$
- (c) Use lower rank send first and higher rank receive first method to exchange data, therefore avoid deadlock.
- (d) Conduct sum operation to add the received data to local data elementwise in place.

Assign local data to global array.

2. Many2Many algorithm:

According to the data structure, the message sizes are similar to each other. So that we decide not to apply a data split process, as this takes additional computation and communication time.

- (a) Used direct mapping route to send and receive the message size.
- (b) Allocate memory for the data to be sent and received.
- (c) Use the cyclic shift route to broadcast the data to all other processors.

3. Theoretical runtime analysis:

- (a) For AllReduce Algorithm: Computation (Let m be the length of data in local processors)
- $\log p$ iteration and m sum operation:

$$\mathcal{O}(m \log p)$$

communication - $\log p$ iteration and m number of data to transfer:

$$\mathcal{O}(\tau \log p + \mu m \log p)$$

- (b) For Many2Many: Computation - For computation, only applied single for loop to calculate the message sizes:

$$\mathcal{O}(p)$$

Computation - Used one direct mapping for a single value and a cyclic shift to send and receive the data, with message size m :

$$\mathcal{O}(\tau p^2 + \mu m p)$$

4. Runtime plots:

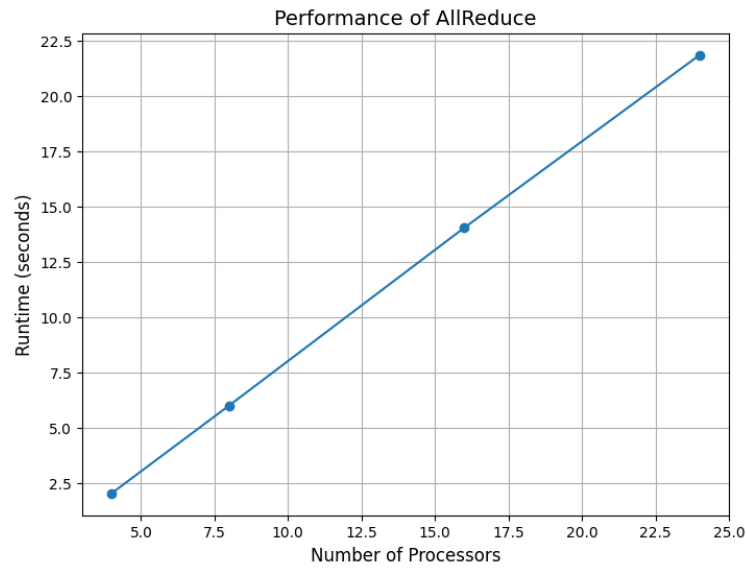


Figure 1: Runtime comparison between AllReduce

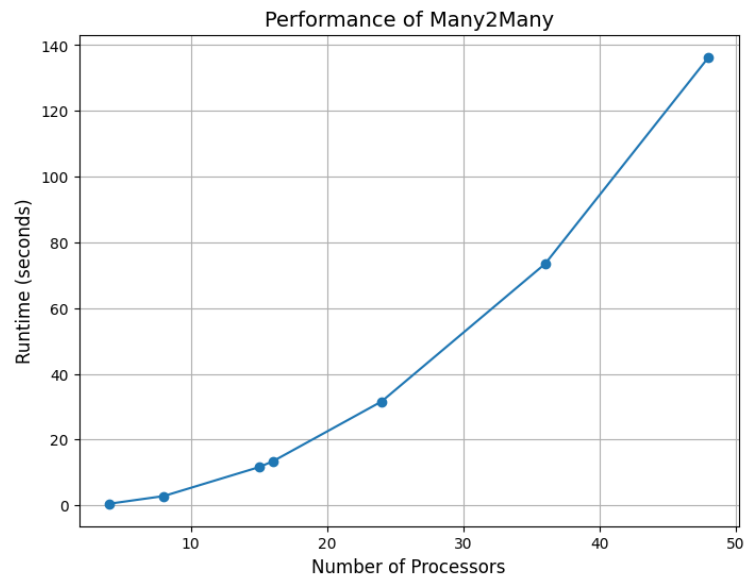


Figure 2: Runtime comparison between Many2Many

In the above plots, we compare the performance of two custom collective algorithms: an

AllReduce implementation using a hypercubic permutation and a Many-to-Many algorithm using an arbitrary permutation.

For AllReduce, the runtime grows from about 2.0 s at 4 processors to roughly 22.0 s at 24 processors. .

By contrast, the Many2Many algorithm exhibits a steeper increase in runtime, starting at about 0.45 s for 4 processors and reaching nearly 32.0 s for 24 processors.

These results demonstrate that, as p grows, the hypercube-based AllReduce approach scales more efficiently than the arbitrary-permutation Many2Many method.

Though our theoretical analysis matches the many2many (may contribute to a large latency), there shows a little deviation with AllReduce plots, we have two assumption:

1. the runtime will eventually converge to the theoretical value as the current processor number is not large enough to show a curve tendency.
2. the runtime is not only determined by our algorithm, the HPC environment may not be ideal.